Tensor-Product $N \times N$ Gauss–Legendre Pixel Integration

Renderer Convergence Conjecture — Experiment 1

July 17, 2026

1 Purpose and convention

This note specifies the $N \times N$ Gauss–Legendre rule used by the Experiment 1 custom image integrator and suitable for a Riley pixel-box integration mode. It approximates the *area average* of a scalar texture/intensity field $I(x, y)$ over one axis-aligned pixel

$$P = [x_0, x_0 + \Delta x] \times [y_0, y_0 + \Delta y].$$

The desired camera value is

$$\bar{I}_P = \frac{1}{\Delta x \Delta y} \int_{y_0}^{y_0+\Delta y} \int_{x_0}^{x_0+\Delta x} I(x, y) \, \mathrm{d}x \, \mathrm{d}y. \tag{1}$$

For a square camera pixel, set $\Delta x = \Delta y = h$. The rule has N^2 evaluations per output pixel. Here N means the number of nodes *per direction*, not the total number of samples.

2 One-dimensional Gauss–Legendre rule

Let (ξ_i, w_i), $i = 1, \ldots, N$, be the standard Gauss–Legendre nodes and weights on $[-1, 1]$:

$$\int_{-1}^{1} f(\xi) \, \mathrm{d}\xi \approx \sum_{i=1}^{N} w_i f(\xi_i). \tag{2}$$

It is exact for polynomials through degree $2N - 1$. The nodes are the roots of the Legendre polynomial P_N, and the weights may be computed as

$$w_i = \frac{2}{(1 - \xi_i^2)[P_N'(\xi_i)]^2}. \tag{3}$$

In practice, use a trusted Gauss–Legendre routine (for example `numpy.polynomial.legendre.leggauss(N)`) or a precomputed table.

3 Map the rule to one pixel

Map each reference node independently to the physical pixel:

$$x_i = x_0 + \frac{\Delta x}{2}(1 + \xi_i), \tag{4}$$

$$y_j = y_0 + \frac{\Delta y}{2}(1 + \xi_j). \tag{5}$$

The Jacobian for the two-dimensional map is $\Delta x \Delta y / 4$. Therefore the *integral* is approximated by

$$\int_P I(x, y) \, \mathrm{d}x \, \mathrm{d}y \approx \frac{\Delta x \Delta y}{4} \sum_{j=1}^{N} \sum_{i=1}^{N} w_i w_j I(x_i, y_j). \tag{6}$$

Dividing by the pixel area gives the form to use for a camera pixel average:

$$\boxed{\bar{I}_P^{(N)} = \sum_{j=1}^{N}\sum_{i=1}^{N} \widehat{w}_i \widehat{w}_j I(x_i, y_j), \qquad \widehat{w}_i = \frac{w_i}{2}.} \tag{7}$$

The normalised weights sum to one:

$$\sum_{i=1}^{N} \widehat{w}_i = 1, \qquad \sum_{j,i} \widehat{w}_i \widehat{w}_j = 1.$$

Consequently a constant texture remains exactly constant, a useful implementation check.

4 Implementation recipe

1. Generate ξ_i, w_i once for the selected order N.
2. Form offsets from the pixel lower-left corner:

$$d_i = \frac{h}{2}(1 + \xi_i), \qquad \widehat{w}_i = \frac{w_i}{2}.$$

3. Form the tensor product. For every pair (i, j), use sample offset (d_i, d_j) and weight

$$W_{ji} = \widehat{w}_j \widehat{w}_i = \frac{w_j w_i}{4}.$$

4. For each output pixel, evaluate the texture at its N^2 sample positions and accumulate $\sum_{j,i} W_{ji} I_{ji}$.

Equivalent pseudocode is:

```
xi, w = gauss_legendre(N)       # nodes/weights on [-1, 1]
offset = 0.5 * (xi + 1) * h
weight = 0.5 * w

value = 0
for j in range(N):
    for i in range(N):
        sample_xy = pixel_lower_left + (offset[i], offset[j])
        value += weight[i] * weight[j] * shade(sample_xy)
```

The Experiment 1 Python implementation uses exactly this construction: `offsets = 0.5*(points+1)*pixel_s` `wts_norm = weights*0.5`, and the outer product `wts_norm[:, None] * wts_norm[None, :]`. It therefore produces a pixel *average*; no further division by h^2 is required.

5 Riley integration semantics

For an alternative Riley pixel-box integrator, the quadrature must be applied to the same continuous camera-pixel footprint as the current SSAA rule. For each Gauss point:

1. construct the corresponding camera ray/sample position;
2. perform the normal Riley geometry/inverse-map and visibility work at that point;

3. evaluate the selected shader there; and

4. add the result with tensor-product weight W_{ji}.

The final resolved pixel is the weighted sum, not the arithmetic mean. An arithmetic mean is correct only for equal-weight SSAA samples. If a sample is outside the image/mesh, it must follow the existing renderer's background and coverage convention; its quadrature weight must not be silently renormalised unless that convention explicitly defines a conditional average over covered area.

6 Low-order checks

N	nodes ξ_i	weights w_i
1	0	2
2	$\pm 1/\sqrt{3}$	$1, 1$
3	$0,\ \pm\sqrt{3/5}$	$8/9,\ 5/9,\ 5/9$

Thus 1×1 Gauss is the pixel-centre sample. A 2×2 rule uses points at $h(1 \pm 1/\sqrt{3})/2$ from the lower-left corner and assigns every point weight $1/4$. Higher orders have unequal weights and are therefore not interchangeable with regular-grid SSAA.

7 Accuracy and cost

The tensor-product rule is polynomial-exact up to degree $2N - 1$ in each coordinate. For the Experiment 1 eggbox, the integrand is trigonometric, so the rule is not finite-order exact but converges rapidly as N grows. The cost is N^2 shader/geometry evaluations per output pixel. Compare it against the analytic eggbox box average to choose the lowest adequate order. The existing convergence parameter for Gauss integration is consequently $S = N^2$, matching the total point count reported for SSAA.

8 Computing nodes and weights for arbitrary N

For a production renderer, precompute the one-dimensional rule once per requested order N, cache it, and form the two-dimensional tensor product from it. Do *not* derive the two-dimensional nodes independently. Two standard arbitrary-order constructions are suitable.

8.1 Golub–Welsch construction

The most direct robust approach is the Golub–Welsch algorithm. Form the symmetric $N \times N$ Jacobi matrix

$$J = \begin{bmatrix} 0 & \beta_1 & & \\ \beta_1 & 0 & \ddots & \\ & \ddots & \ddots & \beta_{N-1} \\ & & \beta_{N-1} & 0 \end{bmatrix}, \qquad \beta_k = \frac{k}{\sqrt{4k^2 - 1}}, \quad k = 1, \ldots, N-1. \tag{8}$$

If λ_i is the ith eigenvalue and $\mathbf{v}_i$ its unit eigenvector, then

$$\boxed{\xi_i = \lambda_i, \qquad w_i = 2\,[\mathbf{v}_i]_1^2.} \tag{9}$$

The matrix is real symmetric, so use a symmetric tridiagonal eigensolver. This produces ordered nodes and positive weights with excellent numerical stability.

8.2 Newton refinement of Legendre roots

If an eigensolver is unavailable, solve for the roots of P_N directly. Evaluate the Legendre polynomials using

$$P_0(x) = 1, \qquad P_1(x) = x, \tag{10}$$

$$P_{m+1}(x) = \frac{(2m+1)xP_m(x) - mP_{m-1}(x)}{m+1}, \qquad m = 1, \ldots, N-1. \tag{11}$$

At a trial point x, compute the derivative without differentiating the recurrence:

$$P'_N(x) = \frac{N\left(P_{N-1}(x) - xP_N(x)\right)}{1-x^2}. \tag{12}$$

For the kth positive root, a useful initial guess is

$$x_k^{(0)} = \cos\left(\frac{\pi(k-\frac{1}{4})}{N+\frac{1}{2}}\right), \qquad k = 1, \ldots, \left\lceil \frac{N}{2} \right\rceil. \tag{13}$$

Apply Newton iteration until the update is below a floating-point tolerance:

$$x_k^{(r+1)} = x_k^{(r)} - \frac{P_N(x_k^{(r)})}{P'_N(x_k^{(r)})}. \tag{14}$$

After convergence, assign

$$\boxed{\xi_k = x_k, \qquad w_k = \frac{2}{(1-x_k^2)[P'_N(x_k)]^2}.} \tag{15}$$

Use symmetry to fill the paired negative root and identical weight:

$$\xi_{N+1-k} = -\xi_k, \qquad w_{N+1-k} = w_k.$$

For odd N, include the central node $\xi = 0$ only once.

8.3 Required validation checks

For any generated rule, verify numerically that

$$-1 < \xi_i < 1, \qquad w_i > 0, \qquad \sum_{i=1}^{N} w_i = 2.$$

For the pixel-average rule, the corresponding two-dimensional weights must sum to one:

$$\sum_{j=1}^{N}\sum_{i=1}^{N} \frac{w_i w_j}{4} = 1.$$

These checks catch incorrect Jacobian factors, root ordering errors, and accidental use of integral weights where average weights are required.